1/24/2025 Friday	9 AM - 10 AM	Advance SQL Query

Evaluation of previous class

1/24/2025	10 AM - 11 AM	Union & Union All

In SQL Server, the UNION and UNION ALL operators are used to combine the results of two or more SELECT queries into a single result set. However, there are key differences between them:

1. UNION

- **Purpose:** Combines the results of two or more SELECT queries and removes duplicate rows.
- **How it works:** It automatically eliminates duplicate rows from the final result set.
- **Performance:** Generally slower than UNION ALL because SQL Server has to perform additional work to remove duplicates.

Example:

sql

SELECT column1 FROM table1

UNION

SELECT column1 FROM table2;

In this example, the result will include only distinct values from both table1 and table2 for column1.

2. UNION ALL

- **Purpose:** Combines the results of two or more SELECT queries, including **all rows**, even if they are duplicates.
- **How it works:** It does not remove duplicates, so if both SELECT statements return the same row, the result set will include that row multiple times.

- **Performance:** Generally faster than UNION because SQL Server does not need to check for duplicates.

Example:

sql

SELECT column1 FROM table1

UNION ALL

SELECT column1 FROM table2;

In this example, the result will include all rows from table1 and table2 for column1, including duplicates.

Key Differences:

Aspect	**UNION**	**UNION ALL**
Duplicates	Removes duplicates	Includes duplicates
Performance	Slower due to duplicate check	Faster (no duplicate check)
Use case	When you want unique rows	When duplicates are acceptable or needed

When to Use:

- **Use UNION** when you need to combine results and eliminate duplicates.
- **Use UNION ALL** when you need all rows, including duplicates, and want better performance.

Let me know if you'd like more examples or further clarification!

1/24/2025	11 AM - 12 PM	**Difference between Store Procedure & Function**

Summary Table

Feature	Stored Procedure	Function
Return Type	Does not return a value directly	Returns a single value or a table
Data Modification	Can modify data (INSERT, UPDATE, DELETE)	Cannot modify data
Execution	Executed using `EXEC`	Invoked as part of a query expression
Parameters	Input, output, or input-output	Only input parameters
Transaction Control	Can use transactions (BEGIN, COMMIT, ROLLBACK)	No transaction control
Error Handling	Supports `TRY...CATCH`	Limited error handling capabilities
Use Case	Used for business logic, data modification	Used for computations or returning values in queries

In summary:

- **Stored Procedures** are better for performing actions, especially those that modify the state of the database or involve complex logic.
- **Functions** are more suited for calculations or transformations that can return values or tables,

- **Functions** are more suited for calculations or transformations that can return values or tables, and they are typically used within queries to return a result.

1/24/2025	12PM - 1 PM	Sub-Query

A **subquery** in SQL is a query embedded inside another query (usually a SELECT, INSERT, UPDATE, or DELETE statement) to retrieve data that is used in the main query. Subqueries help simplify complex queries, make them more readable, and allow you to perform operations on the result of another query.

Types of Subqueries

Subqueries can be classified into the following types:

1. **Single-row subquery**:

- Returns a single value (one row, one column).
- Often used in conditions like =, >, <, etc.

Example:

sql

SELECT name, salary

FROM employees

WHERE salary = (SELECT MAX(salary) FROM employees);

2. **Multi-row subquery**:
    - Returns multiple rows (but usually one column).
    - Often used with operators like IN, ANY, ALL.

Example:

sql

SELECT name

FROM employees

WHERE department_id IN (SELECT department_id FROM departments WHERE location = 'New York');

3. **Multi-column subquery**:
    - Returns multiple rows and columns.
    - Can be used in WHERE or HAVING clauses.

Example:

sql

SELECT name, salary

FROM employees

WHERE (department_id, job_id) IN (SELECT department_id, job_id FROM employees WHERE salary > 50000);

4. **Correlated subquery**:

- A subquery that references columns from the outer query. It is executed for each row of the outer query.
- This type of subquery is typically slower because it needs to be evaluated once for each row in the outer query.

Example:

sql

SELECT name, salary

FROM employees e1

WHERE salary > (SELECT AVG(salary) FROM employees e2 WHERE e1.department_id = e2.department_id);

Subquery Placement in SQL Statements

A subquery can be used in several places within SQL queries:

1. **In the SELECT Clause**:
   - Subqueries can return a value to be displayed in the result set.
   - They can be used for calculating values like averages or counts.

Example:

sql

SELECT name,

(SELECT MAX(salary) FROM employees WHERE department_id = e.department_id) AS max_salary

FROM employees e;

2. **In the FROM Clause**:
   - A subquery can be used as a virtual table, providing data that the outer query can operate on.

Example:

sql

```
SELECT avg_salary, COUNT(*)
FROM (SELECT salary FROM employees WHERE department_id = 10) AS dept_salaries;
```

3. **In the WHERE Clause**:
    - A subquery is often used to filter rows based on a condition evaluated by another query.

Example:

```
sql
--------
SELECT name, salary
FROM employees
WHERE department_id = (SELECT department_id FROM departments WHERE name = 'Sales');
```

4. **In the HAVING Clause**:
    - Subqueries can be used in HAVING to filter the results of aggregate functions.

Example:

```
sql
--------
SELECT department_id, AVG(salary)
FROM employees
GROUP BY department_id
HAVING AVG(salary) > (SELECT AVG(salary) FROM employees);
```

Benefits of Using Subqueries

- **Modular Queries**: Breaks down complex queries into simpler parts.
- **Reusable**: A subquery can be reused multiple times within the same query.
- **Improves Readability**: Using subqueries can make SQL queries easier to understand compared to using joins for the same purpose.

Limitations of Subqueries

- **Performance**: Correlated subqueries can be inefficient because the subquery is executed for every row of the outer query.
- **Cannot be Indexed**: Subqueries cannot always take advantage of indexes (especially correlated subqueries).

Examples of Subqueries

Example 1: Subquery in SELECT

Get the names of employees along with the highest salary in their department:

sql

```
SELECT name,
    (SELECT MAX(salary) FROM employees WHERE department_id = e.department_id)
AS highest_salary
FROM employees e;
```

Example 2: Subquery in WHERE

Find employees who earn more than the average salary of all employees:

sql

```
SELECT name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

Example 3: Subquery in FROM

Get the average salary of employees in each department:

sql

```
SELECT department_id, AVG(salary)
FROM (SELECT department_id, salary FROM employees) AS dept_salaries
GROUP BY department_id;
```

Example 4: Correlated Subquery

Find employees whose salary is greater than the average salary of their department:

sql

SELECT name, salary

FROM employees e1

WHERE salary > (SELECT AVG(salary) FROM employees e2 WHERE e1.department_id = e2.department_id);

Summary

- **Subqueries** are nested queries within another query, allowing you to perform more complex data retrieval or comparison.
- Subqueries can be used in various parts of a SQL query (e.g., SELECT, FROM, WHERE, HAVING).
- There are different types of subqueries, including **single-row**, **multi-row**, **multi-column**, and **correlated subqueries**.
- Subqueries provide a way to simplify complex logic but can sometimes be slower due to performance issues, particularly with correlated subqueries.

1/24/2025	2 PM - 3 PM	View

A **view** in SQL Server is a virtual table that is defined by a query. It is essentially a saved SELECT statement that you can treat like a table in subsequent queries. Views allow you to encapsulate complex queries and present them as a simple table, making it easier to retrieve data in a consistent and reusable manner.

Key Characteristics of a View:

- **Virtual Table**: A view doesn't store data physically. Instead, it stores the query definition. When you query a view, the query inside the view definition is executed to retrieve data.
- **Read-Only or Updatable**: Some views are updatable (i.e., you can perform INSERT, UPDATE, or DELETE operations on them), while others are read-only (i.e., you can only select data from them).
- **Simplification**: Views simplify complex queries by encapsulating them and providing a simpler interface.

- **Security**: Views can be used to restrict access to specific columns or rows of data by defining a view that only exposes the required data.

Creating a View

To create a view in SQL Server, you use the CREATE VIEW statement, followed by the view name and the SELECT query that defines the view.

Syntax:

sql

CREATE VIEW view_name AS

SELECT column1, column2, ...

FROM table_name

WHERE condition;

Example:

sql

CREATE VIEW EmployeeDetails AS

SELECT EmployeeID, Name, Salary, DepartmentID

FROM Employees

WHERE Salary > 50000;

This view, named EmployeeDetails, will show employees who have a salary greater than 50,000, along with their EmployeeID, Name, Salary, and DepartmentID.

Querying a View

Once a view is created, you can query it like a regular table:

sql

SELECT * FROM EmployeeDetails;

This query will return all employees from the EmployeeDetails view who have a salary greater than 50,000.

Modifying a View

If you need to change the definition of an existing view, you use the ALTER VIEW statement:

Syntax:

sql

```
ALTER VIEW view_name AS
SELECT new_columns
FROM new_table
WHERE new_condition;
```

Example:

sql

```
ALTER VIEW EmployeeDetails AS
SELECT EmployeeID, Name, Salary, DepartmentID, HireDate
FROM Employees
WHERE Salary > 50000;
```

In this example, the view is modified to also include the HireDate column.

Dropping a View

To remove a view from the database, you use the DROP VIEW statement:

Syntax:

sql

```
DROP VIEW view_name;
```

Example:

sql

```
DROP VIEW EmployeeDetails;
```

This will delete the EmployeeDetails view.

Types of Views

1. **Simple Views**:
    - A simple view is created from a single table and can be updatable.
    - **Example**:

sql

```
CREATE VIEW SimpleView AS
SELECT column1, column2 FROM table_name;
```

2. **Complex Views**:
    - A complex view can be created from multiple tables using JOIN, GROUP BY, or other SQL operations.
    - **Example**:

sql

```
CREATE VIEW ComplexView AS
SELECT e.EmployeeID, e.Name, d.DepartmentName
FROM Employees e
JOIN Departments d ON e.DepartmentID = d.DepartmentID;
```

3. **Updatable Views**:
    - Some views allow data modification (INSERT, UPDATE, or DELETE). These views must meet certain criteria, such as:
        - No DISTINCT, GROUP BY, HAVING, or aggregate functions.
        - No JOIN with multiple tables unless certain rules are followed.
        - The view is based on a single table.

Example of an updatable view:

sql

```
CREATE VIEW UpdatableView AS
SELECT EmployeeID, Name, Salary
```

FROM Employees;

You can perform UPDATE on this view because it is based on a single table without aggregates or joins.

4. **Non-updatable Views**:
    - Views that include complex logic, such as JOIN, aggregate functions, or DISTINCT, may not be directly updatable.
    - **Example**:

sql

CREATE VIEW NonUpdatableView AS

SELECT e.EmployeeID, e.Name, SUM(s.Salary) AS TotalSalary

FROM Employees e

JOIN Salaries s ON e.EmployeeID = s.EmployeeID

GROUP BY e.EmployeeID, e.Name;

Benefits of Using Views

1. **Simplified Queries**: Complex queries can be encapsulated into views, making it easier to reuse them in other queries without re-writing the complex logic.
2. **Security**: Views can be used to expose only specific columns or rows of a table, restricting access to sensitive data.
3. **Abstraction**: Views abstract the complexity of the underlying database structure, presenting a simplified interface for users.
4. **Reusability**: A view can be reused in multiple queries, promoting code reusability.

Limitations of Views

1. **Performance**: Since views are not materialized (i.e., they do not store data physically), complex views can lead to performance issues, especially if the view is queried frequently and involves large datasets.
2. **Updatability**: Not all views are updatable. Some views, especially those based on multiple tables or those that involve aggregation, cannot be updated directly.
3. **No Indexing**: Views do not support indexing. However, indexed views (also called **materialized views**) are available in SQL Server under specific conditions.

Indexed Views (Materialized Views)

In SQL Server, you can create an **indexed view** (a view with a clustered index) that stores the result set physically, improving performance for certain types of queries. These views are sometimes referred to as **materialized views** in other databases.

Syntax for creating an indexed view:

1. First, create a view as usual.
2. Then, create a clustered index on that view.

Example:

sql

CREATE VIEW SalesSummary AS

SELECT ProductID, SUM(SalesAmount) AS TotalSales

FROM Sales

GROUP BY ProductID;

-- Create a clustered index on the view

CREATE UNIQUE CLUSTERED INDEX idx_SalesSummary ON SalesSummary(ProductID);

Summary of View Operations

Operation	SQL Command	Description
Create a View	CREATE VIEW view_name AS SELECT ...	Creates a new view based on a SELECT query.
Modify a View	ALTER VIEW view_name AS SELECT ...	Modifies an existing view.
Drop a View	DROP VIEW view_name;	Deletes a view from the database.
Query a View	SELECT * FROM view_name;	Retrieves data from a view, just like querying a table.

Conclusion

Views in SQL Server are powerful tools for abstracting complex queries, simplifying data access, enhancing security, and ensuring code reusability. While views are generally read-only, they can also be updatable if created under the right conditions. Always be mindful of performance considerations when using complex views, especially if they involve large datasets or frequent queries.

1/24/2025	3 PM - 4 PM	Generate XML file from Table

In SQL Server, you can generate an **XML file** from a table using the FOR XML clause. This allows you to format the results of a query as XML, which can be useful for exporting data or interacting with other systems that require XML data.

There are three modes of FOR XML that can be used:

1. **RAW**
2. **AUTO**
3. **EXPLICIT**
4. **PATH**

Here's how each mode works and how you can generate XML from a table:

1. FOR XML RAW

- **Description**: This mode generates a raw XML output, where each row in the result set is wrapped in an XML element with attributes corresponding to column names.

Example:

sql

SELECT EmployeeID, Name, Salary

FROM Employees

FOR XML RAW;

This query will return XML like this:

xml

<row EmployeeID="1" Name="John Doe" Salary="50000" />

<row EmployeeID="2" Name="Jane Smith" Salary="60000" />

2. FOR XML AUTO

- **Description**: This mode generates XML in a hierarchical structure where the table name becomes the root element, and each row is represented as a child element with nested elements for each column.

Example:

sql

SELECT EmployeeID, Name, Salary

FROM Employees

FOR XML AUTO;

This query will return XML like this:

xml

<Employees EmployeeID="1" Name="John Doe" Salary="50000" />

<Employees EmployeeID="2" Name="Jane Smith" Salary="60000" />

3. FOR XML EXPLICIT

- **Description**: This mode is more complex and allows you to define the XML structure explicitly by using numeric codes for each column and level in the hierarchy.

Example:

sql

SELECT 1 AS Tag, NULL AS Parent, EmployeeID AS [Employee!1!EmployeeID], Name AS [Employee!1!Name], Salary AS [Employee!1!Salary]

FROM Employees

FOR XML EXPLICIT;

This query generates an XML with the structure that you define manually. The EXPLICIT mode requires more detailed management of the XML hierarchy.

4. FOR XML PATH

- **Description**: This mode allows you to define custom element names and hierarchical structure. It is the most flexible and commonly used method for generating XML.

Example 1: Simple XML with Custom Element Names

sql

SELECT EmployeeID AS [EmployeeID], Name AS [EmployeeName], Salary AS [EmployeeSalary]

FROM Employees

FOR XML PATH('Employee');

This query will return XML like this:

xml

<Employee>

  <EmployeeID>1</EmployeeID>

  <EmployeeName>John Doe</EmployeeName>

  <EmployeeSalary>50000</EmployeeSalary>

</Employee>

<Employee>

  <EmployeeID>2</EmployeeID>

  <EmployeeName>Jane Smith</EmployeeName>

  <EmployeeSalary>60000</EmployeeSalary>

</Employee>

Example 2: Nested XML with Custom Element Names

You can also nest XML data within elements, making it more hierarchical.

sql

SELECT DepartmentID, DepartmentName,

```
    (SELECT EmployeeID, Name, Salary
     FROM Employees
     WHERE Employees.DepartmentID = Departments.DepartmentID
     FOR XML PATH('Employee')) AS EmployeesXML
FROM Departments
FOR XML PATH('Department');
```

This query will return XML like this:

```
xml
--------
<Department>
  <DepartmentID>1</DepartmentID>
  <DepartmentName>HR</DepartmentName>
  <EmployeesXML>
    <Employee>
      <EmployeeID>1</EmployeeID>
      <Name>John Doe</Name>
      <Salary>50000</Salary>
    </Employee>
    <Employee>
      <EmployeeID>2</EmployeeID>
      <Name>Jane Smith</Name>
      <Salary>60000</Salary>
    </Employee>
  </EmployeesXML>
</Department>
<Department>
  <DepartmentID>2</DepartmentID>
  <DepartmentName>Finance</DepartmentName>
```

```
  <EmployeesXML>
    <Employee>
      <EmployeeID>3</EmployeeID>
      <Name>Michael Johnson</Name>
      <Salary>75000</Salary>
    </Employee>
  </EmployeesXML>
</Department>
```

5. FOR XML PATH with Concatenation

You can also concatenate the results into a single XML element.

Example:

sql

```
--------
SELECT STUFF((
  SELECT ',' + Name
  FROM Employees
  FOR XML PATH('')
), 1, 1, '') AS EmployeeNames;
```

This query will return a single string of employee names concatenated as comma-separated values, which can be useful if you want to format the data for further use.

6. Saving XML to a File

In SQL Server, generating XML through FOR XML will return XML as part of the result set. To save it to a file, you will typically do this via a SQL Server tool like SQL Server Management Studio (SSMS) or programmatically through the SQL Server API (like ADO.NET).

Here's an example of how to use FOR XML and save the result to an XML file through a stored procedure or SQL Server Management Studio:

Example:

sql

```
DECLARE @XML XML;


SELECT @XML = (
  SELECT EmployeeID, Name, Salary
  FROM Employees
  FOR XML PATH('Employee')
);


-- Output XML to a file (this requires SQL Server Agent or another method to export the result set)
-- You could use BCP, PowerShell, or other methods to save the content of @XML to a file.
SELECT @XML;
```

To save this XML to a file, you could use **SQL Server Integration Services (SSIS)**, **BCP utility**, or **PowerShell**.

Summary

- **RAW**: Generates a flat XML with each row as a separate element.
- **AUTO**: Automatically generates XML in a tabular format based on table names.
- **EXPLICIT**: Allows you to manually define the XML structure.
- **PATH**: The most flexible option for generating custom, hierarchical XML structures.

1/24/2025	4 PM - 5 PM	Load XML file in Table

To load an **XML file** into a SQL Server table, you can use several methods. The most common way is to use the OPENXML function, XML data type, or BULK INSERT with XML files. Below, I'll explain the common methods in detail:

```
DECLARE @xml XML;

-- Load XML from file into the variable
SELECT @xml = BulkColumn
FROM OPENROWSET(BULK 'C:\test\employee1.txt', SINGLE_BLOB) AS x;

-- Insert data into the Employees table
INSERT INTO Employees (EmployeeID, EmployeeName, Department)
SELECT
   Employee.value('EmployeeID[1]', 'INT'),
   Employee.value('EmployeeName[1]', 'VARCHAR(100)'),
   Employee.value('Department[1]', 'VARCHAR(50)')
FROM @xml.nodes('/Employees/Employee') AS T(Employee);

select * from Employees
```

1/24/2025	5 PM - 6 PM	Pracitce Session
1/24/2025	6 PM - 7 PM	Practice Session